ChatGPT

Executive Summary

Integrating diverse source data (CSV/Excel files, HL7 v2 messages, FHIR resources, etc.) into an openEHR clinical repository requires specialized conversion tools and mapping frameworks. In our review we identified several open-source and commercial solutions for this task. Open-source projects like **openEHR_FLAT_Loader** and **FLATEHR** provide Python-based ETL scripts that take tabular/JSON/XML input and generate openEHR compositions via user-defined mappings [1] [2]. The **openFHIR** engine (Apache-2.0) implements the FHIRConnect mapping spec to enable bidirectional conversion between openEHR compositions and FHIR resources [3]. In the commercial space, tools such as **MapEHR** offer YAML-driven legacy data migration workflows, while general integration platforms (e.g. Mirth Connect, NextGen Connect) are often used with custom coding for openEHR import [4] [5]. For each tool we detail the developer/vendor, licensing model, supported input/output formats, mapping approach (GUI or config language), template support, deployment options, maturity, and documentation. We also summarize openEHR community resources (e.g. official SDKs, CKM, EHRBase, Ocean Informatics tools) that underpin these conversions. A comparison table contrasts the tools by license, input formats, mapping approach and deployment.

Conversion Tools

openEHR_FLAT_Loader

Developer/Vendor: Zukunftslabor Gesundheit (Germany) [1]
License: GPL-3.0 (Open Source) [6] [1]
Primary Use Case: ETL from tabular data (e.g. clinical trial CSVs, SQL tables) into openEHR compositions. Focused on small-scale or clinical data exports [7].
Supported Inputs: CSV (semicolon-delimited) and other tabular data (SQL). Also supports exporting to CSV via reverse mapping [8].
Supported Outputs: openEHR Composition instances (conforming to an operational template) in "flat JSON" (simSDT) format for upload via REST to an openEHR server (e.g. EHRBase) [1] [8].
Mapping Approach: The tool generates an Excel mapping table (one column of template paths, one of source columns) and uses manual user entry of mappings [9]. Users run a CLI script (`Step1_Generate_Mapping_File`) which scaffolds this mapping file, fill in how source fields map to the template's flat paths, then run the build/upload step [10] [11].
Archetype/Template Support: Uses openEHR Operational Templates (OPT files) and Web Templates; it does not support arbitrary new archetypes on its own. The mapping generator reads a given OPT/WebTemplate and creates mapping skeletons accordingly [12].
Deployment: Python command-line scripts; runnable on any OS with Python. Configuration is via `config.ini`. Optionally run via provided `.bat` wrappers. Requires an openEHR server (like EHRBase) with flat format support [13] [14]. Docker is not provided, but it can be containerized.
Languages/Dependencies: Python (tested on Python 3); uses libraries for openEHR JSON and HTTP calls.
Maturity: ~392 commits; active development (latest commit May 2024 [15]). Experimental; noted as best suited to simpler datasets (e.g. clinical trial data) [16]. Community activity is modest (GitHub stars=2).
Documentation/Links: GitHub repo with README, how-to guides and sample data [8] [10]. Source: [1] [6].
Pricing: Free (open source).
Strengths/Limitations: Good for structured tabular data with fixed format. Limitations include manual

mapping effort and less suitability for complex, high-frequency sensor streams or very large datasets [16]. Mapping UI is Excel-based, which can help non-developers but may become unwieldy for large templates.

FLATEHR

Developer/Vendor: CRS4 (Italy) [2]
License: GPL-3.0 (Open Source) [17]
Primary Use Case: Low-code mapping of XML/JSON data sources into openEHR compositions (flat format). Targeted for arbitrary JSON/XML, not limited to FHIR or HL7.
Supported Inputs: XML and JSON documents (via XPath/JSONPath extraction). CSV not directly mentioned.
Supported Outputs: openEHR compositions in flat JSON (simSDT) format (ordered key-value pairs per flat template) [18].
Mapping Approach: User writes a YAML configuration file specifying mappings from source keys (XPath/JSONPath) to openEHR flat composition paths [19]. The tool can generate a skeleton YAML from a Web Template. It supports value transforms (Jinja templates, value maps), null flavours, and repeated elements [20] [19].
Archetype/Template Support: Uses Web Templates (.json) for simSDT. Can auto-generate a flat template skeleton from a Web Template for mapping purposes [21] [19].
Deployment: Available as a Python package (`pip install flatehr`) or Docker image (`crs4/flatehr`) [22] [23]. Runs on any OS with Python.
Languages/Dependencies: Python 3 (uses Poetry). Relies on `jmespath` for JSON querying, `lxml` for XML, and an openEHR-flat JSON library for output.
Maturity: ~286 commits; actively maintained (commits as recent as Feb 2025 [24]). Tests and CI included. GitHub stars=16.
Documentation/Links: GitHub (README with examples, config syntax) [25]. Docs folder includes usage examples and schema. Source: [18] [26].
Pricing: Free (open source).
Strengths/Limitations: Flexible (N:N mappings, value transformations, defaulting, nullFlavour). Particularly well-suited to mid-sized integration projects where input is JSON/XML. No built-in GUI; users edit YAML. Does not natively handle CSV or HL7 v2 without preprocessing.

openFHIR

Developer/Vendor: openFHIR community (originally Medblocks, now openFHIR.org with contributions from Karkinos, HiGHmed, Syntaric, etc.) [27]
License: Apache 2.0 (Open Source) [28]
Primary Use Case: Bi-directional FHIR ↔ openEHR mapping engine compliant with the FHIR Connect specification [3]. Often used to expose an openEHR repository via a FHIR API or vice versa.
Supported Inputs: HL7 FHIR Resources (FHIR-R4) and openEHR JSON compositions (canonical/openEHR REST format) [3]. Also supports bulk messaging via queues.
Supported Outputs: The counterpart format: from FHIR to openEHR compositions (flat JSON) or openEHR to FHIR Resources [3]. Data is not stored by openFHIR; it transforms and passes through.
Mapping Approach: Based on **FHIR Connect** (an open mapping specification in YAML) which allows a single mapping config for both directions [3]. Users define mappings in YAML (paths and transformations). The engine provides a REST API and can be invoked on messages or via queue. There is also an "Atlas" UI for managing mappings.
Archetype/Template Support: Requires openEHR templates (Web Templates or OPTs) and corresponding openEHR archetypes. It does not generate archetypes. Works with canonical compositions; no direct support for flat format.

Deployment: Provided as a Dockerized Java Spring Boot service [29]. Supports standalone or with Docker Compose (MongoDB). A public sandbox is available for testing [30].
Languages/Dependencies: Java 17 (Spring Boot), MongoDB (as per default config).
Maturity: Active development; official 2.x series (latest 2.0.5 released Mar 23, 2026) [31]. The older MedBlocks repository was archived in Mar 2026, but development continues at openFHIR/openfhir. Issues and releases are managed on GitHub.
Documentation/Links: GitHub repo with README and links to docs [3]. Official docs at open-fhir.com/documentation.
Pricing: Free (Apache-2.0 open source). There is also an Enterprise edition (Medblocks) with additional features (terminology integration, UI, etc.) under commercial license [32].
Strengths/Limitations: Rigorous FHIR<->openEHR mappings following a published spec. Handles full resource conversions including extensions. Requires writing FHIR Connect mappings (non-trivial). Does **not** handle non-FHIR inputs (e.g. raw CSV/HL7v2), only FHIR. Setup involves Docker/MongoDB. Good for systems needing true bidirectional sync.

ALUR (AQL-based Logical Unified Routing)

Developer/Vendor: Michael Anywar (Tallinn Univ. of Tech / independent) [33]
License: MIT (Open Source) [33]
Primary Use Case: Extracting data from openEHR via AQL and converting it to FHIR format for downstream systems. Essentially "openEHR → FHIR" exporter.
Supported Inputs: Queries against any openEHR repository (via AQL).
Supported Outputs: FHIR R4 Resources (Patient, Observation, etc.) built from openEHR data [34]. Also flexible support for FHIR profiles and extensions.
Mapping Approach: Modular pipeline; uses AQL to select data and custom mapping logic to create FHIR JSON. Mappings are specified in configuration (not detailed in the source). Emphasizes handling of FHIR extensions.
Archetype/Template Support: Works off any openEHR templates; it does not require a specific template as long as AQL can retrieve the data.
Deployment: Java-based backend service (likely Spring). The tools currently run as standalone services (can be Dockerized).
Languages/Dependencies: Java, Apache Camel (for routing), an openEHR Java client.
Maturity: Announced Jun 2025; active development. Code available on GitHub [35].
Documentation/Links: News article with project description [34]; GitHub repo link provided there.
Pricing: Free (MIT).
Strengths/Limitations: Focused on openEHR→FHIR use case with scheduler support. Not applicable for feeding data *into* openEHR. Complements conversion tools by handling the inverse direction with advanced mapping of extensions.

CohortMailer

Developer/Vendor: Michael Anywar (Tallinn Univ. of Tech / independent) [33]
License: MIT (Open Source) [33]
Primary Use Case: Automated extraction of openEHR patient cohorts to CSV reports. Essentially "openEHR → CSV" batch exporter.
Supported Inputs: Queries (AQL) on an openEHR CDR for specific patient groups or criteria.
Supported Outputs: Tabular CSV files (or similar) for the queried data [36].
Mapping Approach: Runs scheduled jobs that query the CDR, filter records by cohort definitions, and format results into CSV. Configured via pre-defined logic (likely YAML/JSON config).
Archetype/Template Support: Uses whatever templates are in the repository; user defines which template fields to include in the CSV output.

Deployment: Python or Java backend service (implemented with cron scheduling).
Languages/Dependencies: Likely Python or Java (not explicitly stated), using openEHR client and CSV libraries.
Maturity: Announced Jun 2025; active development with MIT license.
Documentation/Links: Blog announcement [36] ; code on GitHub (linked in article).
Pricing: Free (MIT).
Strengths/Limitations: Useful for research data extraction, quality reporting, etc. Not designed for ingestion into openEHR, only for exports. Depends on AQL and cohort definitions.

MapEHR

Developer/Vendor: Medical Ltd (Slovenia), mapehr.com (also a partner of openEHR) [5]
License: Proprietary (Paid SaaS/On-premise)
Primary Use Case: Legacy EHR data migration to openEHR (bulk import of existing clinical records).
Supported Inputs: Various legacy formats (likely HL7v2, CSV exports, proprietary database tables, etc.). Specifics not documented publicly, but implies general legacy EHR data.
Supported Outputs: openEHR compositions/EHR entries (using vendor's YAML-based mapping). Possibly outputs to an openEHR CDR.
Mapping Approach: Defines mappings in YAML with full programming capabilities (OO-language constructs) embedded [5] . Users script conversion logic inside YAML. Likely supports templating and complex transforms.
Archetype/Template Support: Can ingest multiple data sources into matching templates. Relies on existing template definitions for the target data model.
Deployment: Cloud service or on-prem. Probably Java-based server or microservice offering.
Languages/Dependencies: Not publicly specified (likely Java/.NET backend).
Maturity: Commercial product (likely mature within customer projects). Limited public information (no public repo).
Documentation/Links: Product page (MapEHR.com) and openEHR platform listing [5] . No open docs.
Pricing: Commercial (likely per-project or subscription; contact vendor).
Strengths/Limitations: High flexibility (full programming within mappings). Aimed at large migrations. Proprietary/paid, so source details not transparent. Complexity requires skilled team or vendor support. Supports mixed source formats.

Mirth Connect (NextGen Connect)

Developer/Vendor: NextGen (former Mirth Corporation)
License: GPL-3.0 / AGPL (Community Edition is open source)
Primary Use Case: General healthcare integration engine (routing, transforming HL7/CDA/JSON etc.). Widely used to interface disparate systems, including openEHR.
Supported Inputs: HL7 v2.x messages, HL7 v3/CDA, FHIR, JSON, XML, flat files (CSV, spreadsheets via file reader) and many other formats.
Supported Outputs: Any format via connectors. Can call openEHR REST API to insert compositions or flatten data first. Commonly used to transform inbound HL7/CDA to openEHR JSON.
Mapping Approach: Message transformation is done via JavaScript, Velocity templates or user scripts inside the Mirth interface. A GUI allows channel setup and message maps, but logic is largely coded.
Archetype/Template Support: None built-in; mappings are to openEHR paths. Often uses "flat" (path|field) JSON construction. Requires knowledge of openEHR paths.
Deployment: Runs as a Java servlet (supports Windows/Linux). Configuration via an Eclipse-based or web UI. Docker images available (e.g. "mirthconnect").
Languages/Dependencies: Java; uses its own XML/JSON engines and optionally plugins.
Maturity: Very mature (in use since 2002). Large user community. Open source (AGPL) with enterprise

edition (Mirth Connect OS vs Connect Suite).
Documentation/Links: NextGen Connect (open source community docs), GitHub (mirthealth). Numerous community examples of HL7→openEHR mappings exist.
Pricing: Free (Community). Enterprise support available via NextGen.
Strengths/Limitations: Extremely flexible and robust for HL7 v2 and FHIR integration. Not openEHR-aware out of the box, so requires custom scripting for openEHR formats. Excellent for continuous interfaces, but mapping complexity can be high. Lacks a formal openEHR template editor, relying on manual path mapping [4].

OpenEHR Community & Official Tools

Beyond the above converters, the openEHR ecosystem includes several official and community libraries/tools that support data import and processing. The **openEHR Foundation** provides client SDKs (Java, .NET, Python, JavaScript) for creating compositions from templates in code, but not full ETL pipelines. For example, the openEHR Java SDK can generate Java classes from an OPT and help build composition objects programmatically (often used after initial mapping) [37]. The **Clinical Knowledge Manager (CKM)** is the authoritative repository for archetypes/templates, ensuring mappings target sanctioned models. Products like **EHRbase** (open source) or **OceanEHR** (Ocean Informatics) implement the openEHR REST API and provide back-end services where converted data lands. OpenEHR modelling tools (Archetype Designer, Template Designer, LinkEHR) help define target structures but generally do not do ETL themselves. Terminology services (e.g. Ontoserver) and tools like the **openEHR Toolkit** (CaboLabs) assist in generating CSVs of archetype paths for mapping reference, but are not data-conversion engines. In summary, while openEHR itself focuses on data modeling and storage, community projects (SDKs, format converters, query tools) and implementation platforms (EHRBase, HIP CDR, etc.) provide the plumbing and interfaces needed to ingest data from various sources into openEHR repositories, typically via the tools listed above.

Tool Comparison

Tool	Open Source / Paid	Primary Inputs	Mapping Approach	Deployment	Source / Docs
openEHR_FLAT_Loader	Open source (GPL)	CSV, SQL (tables)	Manual mapping in Excel (flat)	CLI (Python scripts)	GitHub (GPL): zlgesundheit/openEHR_FLAT_Loader [1]
FLATEHR	Open source (GPL)	JSON, XML	YAML config (JSONPath/XPath)	CLI / Docker (Python)	GitHub (GPL): CRS4/flatehr [2]
openFHIR	Open source (Apache)	FHIR R4 JSON, openEHR JSON	YAML (FHIR Connect spec)	Docker (Java)	GitHub (Apache): openFHIR/openfhir [3]

Tool	Open Source / Paid	Primary Inputs	Mapping Approach	Deployment	Source / Docs
ALUR	Open source (MIT)	openEHR AQL queries	AQL-driven / custom logic	Docker/ Service (Java)	GitHub (MIT) – linked from openehr.org news [34]
CohortMailer	Open source (MIT)	openEHR AQL queries	AQL-driven filter logic	Service/ cron (Java/ Python)	GitHub (MIT) – as above [36]
MapEHR	Commercial	Legacy EHR (HL7v2, CSV, DB)	YAML with scripting	Cloud/On-premise	Vendor (Medical Ltd) site and [openehr platform page] [5]
Mirth Connect	Open source (GPL)	HL7v2, HL7v3/ CDA, FHIR, CSV, JSON, etc.	Scripted transforms (JavaScript)	Server (Java, with UI)	NextGen Connect (AGPL) [4]
Other					

Licensing and Pricing

Tool	License	Pricing (if paid)
openEHR_FLAT_Loader	GPL-3.0 (OSS)	Free
FLATEHR	GPL-3.0 (OSS)	Free
openFHIR	Apache 2.0 (OSS)	Free (enterprise support available)
ALUR	MIT (OSS)	Free
CohortMailer	MIT (OSS)	Free
MapEHR	Proprietary	Commercial (vendor quote)
Mirth Connect	AGPL (OSS)	Free (enterprise edition available)

```
flowchart LR
    A[CSV / HL7 / FHIR Data] --> B[Mapping / Transformation Layer]
    B --> C[openEHR Composition (Canonical RM)]
    C --> D[EHR System / openEHR CDR]
```

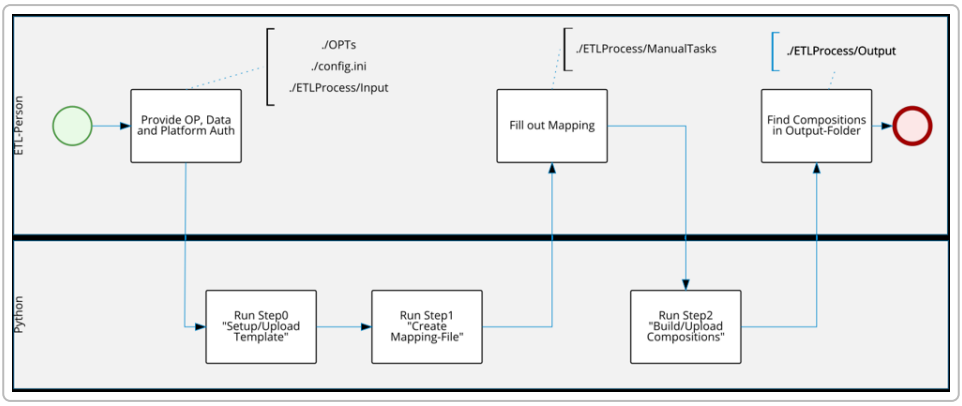

Figure: Example ETL process for openEHR (adapted from the openEHR_FLAT_Loader documentation) showing data extraction and mapping steps leading to composition creation.

---

1 6 7 8 9 10 11 12 13 14 16 GitHub - zlgesundheit/openEHR_FLAT_Loader: Python-based ETL-Scripts to transform tabular source data into the interoperable openEHR-Format using (Excel-)Mappings and CLI. · GitHub
https://github.com/zlgesundheit/openEHR_FLAT_Loader

2 17 18 19 20 21 22 23 25 26 GitHub - crs4/flatehr: tool for generating openEHR compositions from other formats · GitHub
https://github.com/crs4/flatehr

3 27 28 29 30 31 GitHub - openFHIR/openfhir: openFHIR, an engine that implements FHIRConnect specification for bidirectional mapping between openEHR and FHIR · GitHub
https://github.com/openFHIR/openfhir

4 Tools used to transform data source to a canonical JSON - New to openEHR? - openEHR
https://discourse.openehr.org/t/tools-used-to-transform-data-source-to-a-canonical-json/3993

5 Platform – openehr.org
https://openehr.org/platform/

15 Commits · zlgesundheit/openEHR_FLAT_Loader · GitHub
https://github.com/zlgesundheit/openEHR_FLAT_Loader/commits/master/

24 Commits · crs4/flatehr · GitHub
https://github.com/crs4/flatehr/commits/develop/

32 openFHIR - Bridging openEHR & HL7 FHIR
https://open-fhir.com/

33 34 35 36 Bridging the data divide – openehr.org
https://openehr.org/bridging-the-data-divide/

37 Step 4: Load Data | EHRbase Docs
https://docs.ehrbase.org/docs/EHRbase/openEHR-Introduction/Load-Data